

GIT



Índice

1 Introducción al Control de Versiones y Configuración del Entorno	6
1.1 ¿Qué es un Sistema de Control de Versiones (VCS)?	6
1.2 Diferencia entre Git y Plataformas de Alojamiento	6
1.3 Instalación de Git.....	7
1.3.1 Verificación	7
1.3.2 Instalación en macOS	7
1.3.3 Instalación en Windows	7
1.3.4 Instalación en Linux (Ubuntu/Debian)	7
1.4 Configuración Inicial del Entorno.....	8
1.4.1 Configuración de Identidad	8
1.4.2 Configuración de la Rama Principal	8
1.4.3 Configuración del Editor de Texto	8
1.4.4 Gestión de Saltos de Línea (Line Endings)	9
1.5 Verificación de la Configuración.....	9
2 Fundamentos del Repositorio Local.....	10
2.1 Inicialización de un Repositorio.....	10
2.1.1 El directorio	10
2.2 Los Tres Estados de Git	11
2.3 El Ciclo de Trabajo Básico.....	12
2.3.1 Monitorización del Estado	12
2.3.2 Preparación de Cambios (Staging)	12
2.3.3 Confirmación de Cambios (Commit)	12
2.4 Buenas Prácticas en los Mensajes de Commit	13
2.5 Resumen del Flujo	13
3 Gestión de Archivos y Exclusiones	14
3.1 El Archivo	14
3.1.1 Sintaxis y Patrones.....	14
3.1.2 Categorías de Archivos a Ignorar	15
3.1.3 Ejemplo Práctico de	15
3.2 Eliminación de Archivos ()	16
3.2.1 Conservar el archivo localmente ()	16

3.3 Renombrado y Movimiento de Archivos ()	16
4 Inspección y Navegación del Historial	18
4.1 Auditoría del Proyecto:	18
4.1.1 Visualización Condensada	18
4.2 El Identificador SHA-1 (Hash)	19
4.3 Restauración de Archivos (Deshacer cambios locales)	19
4.3.1 1. Descartar cambios en el Directorio de Trabajo	19
4.3.2 2. Sacar archivos del Área de Preparación (Unstage)	19
4.4 Viaje en el Tiempo: Detached HEAD	20
5 Ramificación (Branching) y Fusión (Merging)	21
5.1 Arquitectura de una Rama	21
5.2 Gestión de Ramas	21
5.2.1 Creación de Ramas	21
5.2.2 Cambio de Ramas (Switching)	22
5.2.3 Listado de Ramas	22
5.3 Fusión de Ramas (Merging)	22
5.3.1 1. Fusión Fast-forward (Avance rápido)	22
5.3.2 2. Fusión Recursiva (Merge Commit)	23
5.4 Eliminación de Ramas	23
6 Repositorios Remotos y Sincronización	24
6.1 Protocolos de Conexión	24
6.2 Vinculación de un Repositorio ()	24
6.3 Obtención de un Proyecto Existente ()	25
6.4 Publicación de Cambios ()	25
6.4.1 Configuración del Upstream	25
6.5 Actualización y Descarga (vs)	26
6.5.1 1. : La opción segura	26
6.5.2 2. : La opción directa	26
6.6 Flujo de Trabajo Remoto Estándar	26
7 Estrategias de Flujo de Trabajo: Gitflow	27
7.1 Las Ramas Principales (Long-Lived Branches)	28
7.1.1 (anteriormente)	28
7.1.2	28

7.2 Las Ramas de Soporte (Temporary Branches).....	28
7.2.1 Ramas de Funcionalidad ()	28
7.2.2 Ramas de Lanzamiento ().....	29
7.2.3 Ramas de Parche ().....	29
7.3 Resumen del Ciclo de Vida en Gitflow	30
7.4 Consideraciones Técnicas: El "Merge Commit"	30
8 Colaboración y Resolución de Conflictos.....	31
8.1 La Naturaleza del Conflicto.....	31
8.2 Identificación y Lectura de Conflictos.....	31
8.3 El Protocolo de Resolución	32
8.3.1 Paso 1: Edición	32
8.3.2 Paso 2: Marcado ()	32
8.3.3 Paso 3: Finalización ().....	32
8.4 El Modelo de Pull Request (PR)	33
8.4.1 Flujo de Trabajo con PR	33
9 Recuperación de Errores y Mantenimiento	34
9.1 Refinando el Último Commit ()	34
9.1.1 Modificar el mensaje.....	34
9.1.2 Añadir archivos olvidados	34
9.2 El Botón de "Deshacer":	34
9.2.1 Soft Reset ()	35
9.2.2 Mixed Reset ().....	35
9.2.3 Hard Reset ().....	35
9.3 Almacenamiento Temporal:	35
9.3.1 Guardar en el Stash	36
9.3.2 Recuperar del Stash	36
9.3.3 Listar cambios guardados	36
9.4 Resumen de Comandos de Emergencia	36
10 Glosario y Hoja de Referencia Rápida	37
10.1 Glosario de Términos	38
10.2 Cheatsheet de Comandos	39
10.2.1 Configuración e Inicio	39
10.2.2 Ciclo Básico (Local)	39
10.2.3 Gestión de Ramas	40
10.2.4 Sincronización (Remoto)	40

10.2.5 Inspección y Deshacer	41
10.2.6 Utilidades (Stash)	41

1 Introducción al Control de Versiones y Configuración del Entorno

El desarrollo de software profesional requiere no solo la escritura de código, sino también una gestión eficiente de los cambios que este sufre a lo largo del tiempo. Este capítulo establece las bases teóricas del control de versiones y guía al estudiante a través de la instalación y configuración esencial de Git en su entorno de trabajo local.

1.1 ¿Qué es un Sistema de Control de Versiones (VCS)?

Un Sistema de Control de Versiones es una herramienta de software que registra los cambios realizados en un conjunto de archivos a lo largo del tiempo. Su función principal es permitir recuperar versiones específicas en el futuro.

Aunque es posible utilizar el control de versiones con cualquier tipo de archivo, en el contexto del desarrollo de software es el estándar para gestionar el código fuente. Un VCS robusto permite:

- **Historial completo:** Rastrear cada modificación, quién la realizó y cuándo.
- **Trabajo paralelo:** Permitir que múltiples desarrolladores trabajen en el mismo proyecto simultáneamente sin sobrescribir el trabajo de los demás.
- **Recuperación:** Revertir el proyecto a un estado anterior en caso de errores críticos.

Git es un **Sistema de Control de Versiones Distribuido (DVCS)**. A diferencia de los sistemas centralizados, en Git cada desarrollador tiene una copia completa del historial del proyecto en su máquina local, lo que permite trabajar sin conexión a internet y aumenta la seguridad de los datos.

1.2 Diferencia entre Git y Plataformas de Alojamiento

Es un error común confundir Git con plataformas como GitHub, GitLab o Bitbucket. Es fundamental distinguir sus roles:

- **Git:** Es el **software** que se instala y ejecuta en tu ordenador local. Es la herramienta que gestiona el historial de versiones y crea los registros de cambios. Se utiliza a través de la línea de comandos (terminal) o interfaces gráficas.
- **GitHub/GitLab:** Son **servicios de alojamiento en la nube**. Funcionan como servidores remotos que almacenan una copia de tu repositorio Git. Añaden capas de gestión de proyectos, herramientas de colaboración y visualización web sobre la base técnica de Git.

En resumen: Git es la herramienta; GitHub es el servicio donde se aloja el trabajo realizado con dicha herramienta.

1.3 Instalación de Git

Antes de comenzar, es necesario verificar si Git ya está instalado o proceder a su instalación según el sistema operativo.

1.3.1 Verificación

Abre tu terminal y ejecuta el siguiente comando:

```
git --version
```

Si el sistema devuelve un número de versión (ej. `git version 2.40.0`), la herramienta ya está instalada. De lo contrario, sigue las instrucciones para tu sistema operativo.

1.3.2 Instalación en macOS

La forma más recomendada es mediante las *Xcode Command Line Tools*. Ejecuta en la terminal:

```
xcode-select --install
```

Alternativamente, si utilizas el gestor de paquetes **Homebrew**:

```
brew install git
```

1.3.3 Instalación en Windows

Para Windows, se recomienda descargar el instalador oficial desde git-scm.com.

Nota importante: Durante la instalación, asegúrate de seleccionar la opción que instala **Git Bash**. Esta herramienta proporciona una terminal de emulación BASH (similar a Linux/Mac), que es el estándar en la industria y la que utilizaremos durante el curso.

1.3.4 Instalación en Linux (Ubuntu/Debian)

Utiliza el gestor de paquetes `apt` :

```
sudo apt update  
sudo apt install git
```

1.4 Configuración Inicial del Entorno

Una vez instalado Git, es imperativo configurar las variables de entorno globales. Git necesita saber quién está realizando los cambios para asociarlos correctamente en el historial. Esta configuración se realiza una única vez por máquina.

1.4.1 Configuración de Identidad

Estos datos se incrustarán en cada cambio (commit) que realices. Ejecuta los siguientes comandos sustituyendo los valores por tu nombre real y tu correo electrónico:

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu_email@ejemplo.com"
```

Importante: Es recomendable utilizar el mismo correo electrónico que usarás para registrarte en servicios como GitHub o GitLab, ya que esto facilitará la atribución de tu actividad en dichas plataformas.

1.4.2 Configuración de la Rama Principal

Históricamente, la rama por defecto en Git se llamaba `master`. Actualmente, el estándar de la industria ha migrado hacia el nombre `main`. Para configurar Git de modo que los nuevos repositorios usen `main` por defecto, ejecuta:

```
git config --global init.defaultBranch main
```

1.4.3 Configuración del Editor de Texto

Cuando Git necesite que introduzcas un mensaje (por ejemplo, al hacer un commit sin mensaje en línea), abrirá un editor de texto predeterminado. En muchos sistemas, este editor es **Vim**, el cual tiene una curva de aprendizaje elevada.

Para configurar un editor más accesible, como **VS Code**, ejecuta:

```
git config --global core.editor "code --wait"
```

Nota: Para que esto funcione, debes tener VS Code instalado y el comando `code` accesible en tu `PATH` (en VS Code: `Command Palette` > `Shell Command: Install 'code' command in PATH`).

1.4.4 Gestión de Saltos de Línea (Line Endings)

Los sistemas operativos manejan los finales de línea de forma distinta (Windows usa `CRLF`, mientras que macOS y Linux usan `LF`). Para evitar problemas de formato al trabajar en equipos con diferentes sistemas operativos, configura la conversión automática:

Para usuarios de Windows:

```
git config --global core.autocrlf true
```

Para usuarios de macOS/Linux:

```
git config --global core.autocrlf input
```

1.5 Verificación de la Configuración

Para confirmar que todas las variables se han establecido correctamente, puedes listar la configuración actual con:

```
git config --list
```

Deberías ver una salida similar a esta:

```
user.name=Tu Nombre Completo
user.email=tu_email@ejemplo.com
init.defaultbranch=main
core.editor=code --wait
...
```

Con estos pasos, el entorno local está preparado para comenzar a inicializar y gestionar repositorios, proceso que se detallará en el siguiente capítulo.

2 Fundamentos del Repositorio Local

Una vez configurado el entorno, el siguiente paso es comprender la arquitectura interna de Git. A diferencia de guardar archivos de manera convencional (ej. "Guardar como..."), Git opera mediante un sistema de instantáneas (*snapshots*) gestionadas a través de tres áreas lógicas. Este capítulo detalla el ciclo de vida de los datos y los comandos esenciales para registrar cambios en el historial.

2.1 Inicialización de un Repositorio

Para que Git comience a rastrear un proyecto, es necesario inicializar un repositorio. Este proceso transforma un directorio estándar del sistema operativo en un directorio gestionado por Git.

Sítuate en la carpeta de tu proyecto mediante la terminal y ejecuta:

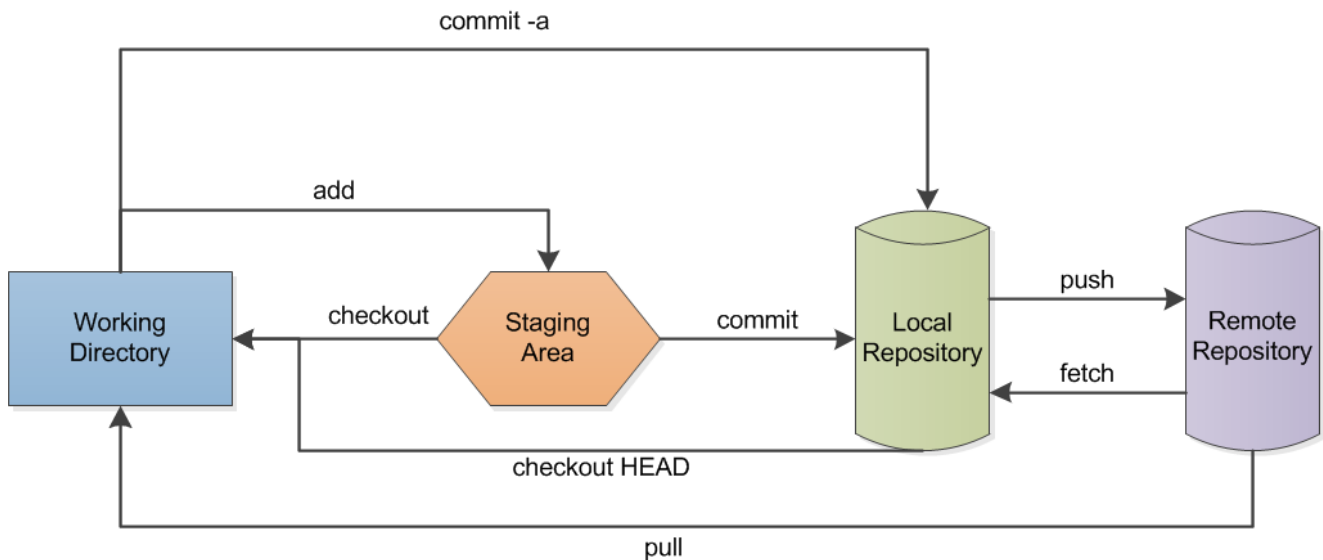
```
git init
```

2.1.1 El directorio `.git`

Al ejecutar el comando anterior, Git crea un subdirectorio oculto llamado `.git`. Este directorio contiene todos los metadatos necesarios para el repositorio: la base de datos de objetos, las referencias a las ramas y el archivo de configuración local.

Advertencia Técnica: El directorio `.git` es el corazón del repositorio. No debe modificarse, eliminarse ni manipularse manualmente bajo ninguna circunstancia, salvo que se tenga un conocimiento avanzado de la estructura interna de Git. Borrar esta carpeta equivale a eliminar todo el historial del proyecto.

2.2 Los Tres Estados de Git



`commit -a`: Directly commit modified and deleted files into the local repository (*no new files!*)

`add`: Add a file to the staging area.

`checkout`: Get a file from the staging area.

`checkout HEAD`: Get a file from the local repository

`commit`: Commit files from the staging area to the local repository

`push`: Send files to the remote repository

`fetch`: Get files from the remote repository

`pull`: Get files from the remote repository and put a copy in the working directory

Para entender cómo Git gestiona los cambios, es fundamental distinguir las tres áreas en las que puede residir un archivo. El flujo de trabajo básico consiste en mover cambios de una área a otra.

1. **Working Directory (Directorio de Trabajo)**: Es la copia de una versión del proyecto en tu disco duro. Aquí es donde editas, borras y creas archivos. Los cambios en esta área **no** están rastreados por Git todavía.
2. **Staging Area (Área de Preparación o Index)**: Es un archivo interno (generalmente llamado *Index*) que almacena información sobre lo que irá en tu próxima confirmación. Funciona como una "zona de carga" donde seleccionas qué cambios específicos deseas guardar.
3. **Repository (Directorio Git / HEAD)**: Es donde Git almacena los metadatos y la base de datos de objetos de manera permanente. Cuando haces un *commit*, los cambios que estaban en el *Staging Area* se guardan aquí como una instantánea inmutable.

2.3 El Ciclo de Trabajo Básico

El flujo estándar de desarrollo sigue una secuencia repetitiva: modificar archivos, prepararlos y confirmarlos.

2.3.1 Monitorización del Estado

El comando más utilizado en Git es `git status`. Este comando no realiza cambios, sino que ofrece un diagnóstico del estado actual del *Working Directory* y del *Staging Area*.

```
git status
```

Este comando informará si los archivos están:

- **Untracked:** Archivos nuevos que Git aún no vigila.
- **Modified:** Archivos rastreados que han sufrido cambios pero aún no están preparados.
- **Staged (Changes to be committed):** Archivos listos para ser confirmados.

2.3.2 Preparación de Cambios (Staging)

Para mover cambios del *Working Directory* al *Staging Area*, utilizamos el comando `git add`. Esto indica a Git que queremos incluir esos cambios en la próxima "foto" del historial.

Para añadir un archivo específico:

```
git add nombre_del_archivo.ext
```

Para añadir todos los archivos modificados y nuevos en el directorio actual:

```
git add .
```

Nota: Es buena práctica revisar con `git status` después de hacer un `git add` para verificar que solo se han añadido los archivos deseados.

2.3.3 Confirmación de Cambios (Commit)

El *commit* es el paso final del ciclo. Toma todo lo que reside en el *Staging Area* y lo guarda permanentemente en el historial del repositorio con un identificador único (hash SHA-1).

El comando requiere obligatoriamente un mensaje que describa los cambios:

```
git commit -m "Descripción breve del cambio realizado"
```

Si se ejecuta `git commit` sin la bandera `-m`, Git abrirá el editor de texto configurado (Capítulo 1) para permitir redactar un mensaje más extenso.

2.4 Buenas Prácticas en los Mensajes de Commit

Un historial de Git limpio y útil depende de la calidad de los mensajes de commit. Un mensaje vago como "arreglos varios" o "cambios" carece de valor semántico para el equipo (y para tu "yo" del futuro).

Se recomienda seguir la convención del **Commit Semántico**:

1. **Imperativo:** Escribir el mensaje como una orden o instrucción (ej. "Añadir validación..." en lugar de "Añadí validación..." o "Añadiendo validación...").
2. **Claro y Conciso:** El resumen no debe exceder los 50-72 caracteres.
3. **Estructura:**
 - `feat` : Para nuevas funcionalidades.
 - `fix` : Para corrección de errores.
 - `docs` : Para cambios solo en documentación.
 - `refactor` : Cambio de código que no arregla bugs ni añade funcionalidades (limpieza).

Ejemplo de un buen commit:

```
git commit -m "feat: añadir sistema de login con validación de email"
```

Ejemplo de un mal commit:

```
git commit -m "nuevos cambios en el login"
```

2.5 Resumen del Flujo

1. Realizas cambios en tus archivos (**Working Directory**).
2. Ejecutas `git status` para ver qué has tocado.
3. Seleccionas los cambios con `git add` (**Staging Area**).
4. Guardas la versión con `git commit` (**Repository**).

3 Gestión de Archivos y Exclusiones

En un entorno de desarrollo profesional, no todos los archivos generados en el directorio de trabajo deben formar parte del historial del proyecto. Archivos de configuración local, dependencias externas, claves de seguridad o binarios compilados no solo añaden "ruido" al repositorio, sino que pueden comprometer la seguridad o inflar innecesariamente el tamaño del proyecto.

Este capítulo aborda cómo definir reglas de exclusión y cómo gestionar correctamente el renombrado y eliminación de archivos dentro de la lógica de Git.

3.1 El Archivo `.gitignore`

El mecanismo principal para evitar que Git rastree archivos no deseados es el archivo `.gitignore`. Este es un archivo de texto plano ubicado en la raíz del repositorio que contiene patrones de búsqueda. Git consultará este archivo antes de determinar si un archivo debe aparecer como *Untracked* en el `git status`.

3.1.1 Sintaxis y Patrones

Cada línea en el archivo `.gitignore` especifica una regla. La sintaxis básica incluye:

- **Comentarios:** Las líneas que comienzan con `#` son ignoradas.
- **Archivos específicos:** Escribir el nombre exacto (ej. `config.local.js`).
- **Directorios:** Terminar el patrón con una barra `/` ignorará todo el contenido de ese directorio (ej. `temp/`).
- **Comodines (Wildcards):** El asterisco `*` coincide con cualquier serie de caracteres. (ej. `*.log` ignorará todos los archivos con extensión `.log`).

3.1.2 Categorías de Archivos a Ignorar

En el desarrollo de software moderno, existen tres categorías críticas que deben incluirse en el `.gitignore`:

1. **Dependencias:** Librerías externas que pueden reinstalarse mediante un gestor de paquetes. No se versionan porque son pesadas y redundantes.
 - Ejemplo: `node_modules/` (Node.js), `vendor/` (PHP/Composer).
1. **Archivos de Entorno y Seguridad:** Archivos que contienen contraseñas, claves API o configuraciones específicas de la máquina del desarrollador. **Versionar estos archivos constituye una grave brecha de seguridad.**
 - Ejemplo: `.env`, `config.json`.
1. **Artefactos de Sistema y Construcción:** Archivos generados automáticamente por el sistema operativo o procesos de compilación.
 - Ejemplo: `.DS_Store` (macOS), `Thumbs.db` (Windows), `dist/`, `build/`.

3.1.3 Ejemplo Práctico de `.gitignore`

A continuación se muestra un ejemplo típico para un proyecto web basado en JavaScript/Node.js:

```
# Dependencias
node_modules/

# Producción
dist/
build/

# Entorno y Seguridad
.env
.env.local

# Logs del sistema
npm-debug.log*
yarn-error.log*

# Sistema Operativo
.DS_Store
```

Nota Técnica: El archivo `.gitignore` **sí** debe ser versionado y confirmado en el repositorio, para que todo el equipo comparta las mismas reglas de exclusión.

3.2 Eliminación de Archivos (`git rm`)

Cuando se elimina un archivo desde el explorador de archivos o mediante el comando `rm` de la terminal, Git detecta que el archivo ha desaparecido del *Working Directory*, pero el archivo sigue existiendo en el registro de Git. `git status` mostrará el cambio como "deleted" pero "not staged".

Para eliminar un archivo de manera correcta y preparar el cambio en un solo paso, se utiliza:

```
git rm nombre_del_archivo.txt
```

Este comando realiza dos acciones:

1. Borra el archivo del sistema de archivos (disco duro).
2. Añade la orden de eliminación al *Staging Area*.

3.2.1 Conservar el archivo localmente (`--cached`)

Es común cometer el error de versionar un archivo que debería haber sido ignorado (como un archivo de configuración o `node_modules`). Si simplemente añades el archivo al `.gitignore` después de haber hecho commit, Git seguirá rastreándolo.

Para dejar de rastrear un archivo en Git pero **mantenerlo** en tu disco duro (para no perder tu configuración local), se utiliza la bandera `--cached` :

```
git rm --cached nombre_del_archivo.env
```

Tras ejecutar esto, el archivo permanecerá en tu carpeta, pero Git lo borrará de su índice en el próximo commit.

3.3 Renombrado y Movimiento de Archivos (`git mv`)

Git no rastrea el movimiento de archivos de forma explícita. Si renombras un archivo manualmente (o lo mueves a otra carpeta), Git lo interpretará como dos eventos separados:

1. La eliminación del archivo con el nombre antiguo.
2. La creación de un archivo nuevo con el nombre nuevo.

Para simplificar este proceso y mantener la coherencia del historial, se utiliza el comando `git mv` :

```
git mv nombre_viejo.txt nombre_nuevo.txt
```

Este comando es una abreviatura que equivale a ejecutar secuencialmente:

1. Renombrar el archivo en el sistema.
2. `git rm nombre_viejo.txt`
3. `git add nombre_nuevo.txt`

Al utilizar `git mv`, Git entiende inmediatamente que se trata de un cambio de nombre, facilitando la lectura del historial en el futuro.

4 Inspección y Navegación del Historial

Una de las características más potentes de un Sistema de Control de Versiones es la capacidad de auditar cada modificación realizada en el proyecto desde su inicio. Git no solo almacena el contenido de los archivos, sino también el contexto de cada cambio (quién, cuándo y por qué).

Este capítulo se centra en cómo leer el registro histórico y cómo revertir cambios locales antes de que sean confirmados definitivamente.

4.1 Auditoría del Proyecto: `git log`

El comando fundamental para revisar la historia del repositorio es `git log`. Al ejecutarlo, Git lista los commits en orden cronológico inverso (del más reciente al más antiguo).

```
git log
```

La salida estándar proporciona cuatro datos críticos por cada commit:

1. **Commit Hash (SHA-1):** El identificador único del cambio (una cadena alfanumérica de 40 caracteres, ej: `9a2b3c...`).
2. **Author:** Nombre y correo del responsable.
3. **Date:** Fecha y hora exacta de la confirmación.
4. **Message:** El texto explicativo introducido por el desarrollador.

Nota de Navegación: Si el historial es extenso, Git utiliza un paginador (habitualmente `less`).

- Utiliza las **flechas** o `Enter` para bajar.
- Presiona la tecla `q` para salir del log y volver a la terminal.

4.1.1 Visualización Condensada

Para obtener una visión general rápida, especialmente en proyectos con un historial largo, es recomendable utilizar banderas de formato:

```
git log --oneline
```

Este comando comprime cada commit en una sola línea, mostrando solo el hash abreviado (los primeros 7 caracteres) y el mensaje del commit.

Para visualizar la estructura del historial, incluyendo futuras ramificaciones, se utiliza:

```
git log --oneline --graph --all
```

4.2 El Identificador SHA-1 (Hash)

Git identifica cada commit mediante un hash SHA-1 (Secure Hash Algorithm 1). Este código se calcula basándose en el contenido de los archivos y en los metadatos del commit. Esto garantiza la integridad del proyecto: si se altera un solo bit de un archivo antiguo o cambia una fecha, el hash cambiaría completamente.

Para referirse a un commit específico en los comandos (por ejemplo, para volver a él), no es necesario escribir los 40 caracteres; generalmente basta con proporcionar los primeros 5 o 7 caracteres, siempre que sean únicos en el repositorio.

4.3 Restauración de Archivos (Deshacer cambios locales)

Durante el desarrollo, es común cometer errores o realizar pruebas que no deben guardarse. Desde la versión 2.23, Git introdujo el comando `git restore` para separar las responsabilidades que antes recaían confusamente en `git checkout` y `git reset`.

Distinguimos dos escenarios de restauración:

4.3.1 1. Descartar cambios en el Directorio de Trabajo

Escenario: Has modificado un archivo, lo has guardado en tu editor, pero te das cuenta de que has roto el código. Quieres que el archivo vuelva a estar exactamente como estaba en el último commit.

```
git restore nombre_del_archivo.ext
```

O para descartar todos los cambios en el directorio actual:

```
git restore .
```

Advertencia Crítica: Este comando es **destructivo**. Los cambios que descartes con `git restore` en el directorio de trabajo no se pueden recuperar, ya que nunca fueron guardados en la base de datos de Git.

4.3.2 2. Sacar archivos del Área de Preparación (Unstage)

Escenario: Has usado `git add` en un archivo por error (por ejemplo, un archivo de configuración que no querías subir), pero quieres mantener tus modificaciones en el archivo para seguir trabajando en él o añadirlo al `.gitignore`.

Para devolver el archivo del *Staging Area* al *Working Directory*:

```
git restore --staged nombre_del_archivo.ext
```

Tras ejecutar esto, el archivo conservará sus modificaciones, pero aparecerá como *Modified* o *Untracked* en `git status`, listo para ser ignorado o modificado de nuevo.

4.4 Viaje en el Tiempo: Detached HEAD

En ocasiones es necesario inspeccionar cómo estaba el proyecto en un punto específico del pasado (por ejemplo, para verificar si un bug existía hace dos semanas).

Para mover el repositorio temporalmente a un commit antiguo:

1. Obtén el hash del commit deseado con `git log`.
2. Ejecuta:

```
git checkout <hash_del_commit>
```

Al hacer esto, entrarás en un estado llamado **"Detached HEAD"** (HEAD desconectado). Esto significa que no estás apuntando a la punta de una rama (como `main`), sino a un commit fijo en el pasado. Puedes mirar los archivos, compilar el código y ejecutar pruebas.

Para volver al presente: Simplemente vuelve a conectar el HEAD a tu rama principal:

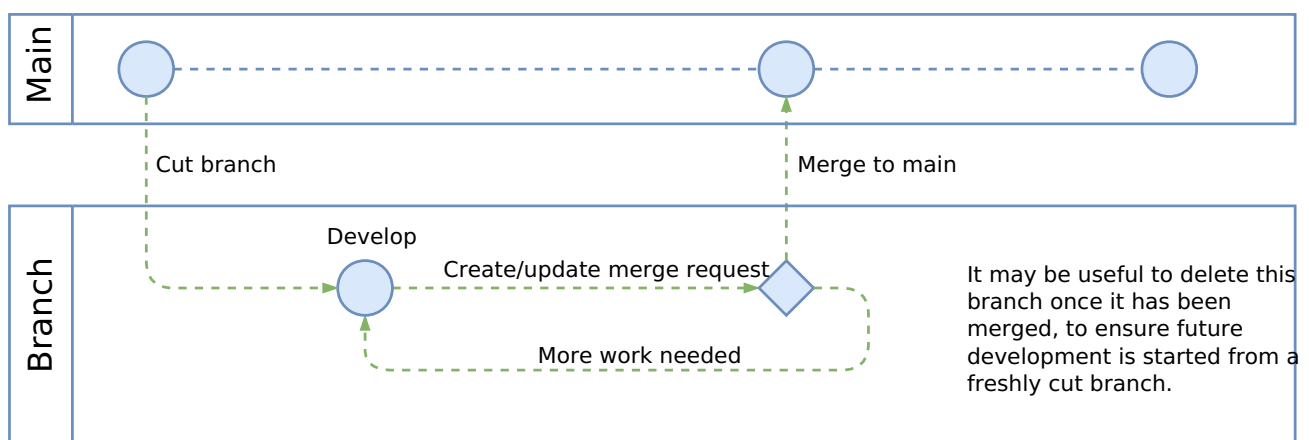
```
git checkout main
```

Nota: Aunque es posible hacer cambios en estado "Detached HEAD", estos se perderán si cambias de rama sin crear una nueva. Se recomienda usar este estado solo para lectura e inspección.

5 Ramificación (Branching) y Fusión (Merging)

La característica más potente de Git es su modelo de ramificación. A diferencia de los sistemas de control de versiones centralizados antiguos, donde crear una rama era una operación costosa y lenta, en Git es un proceso casi instantáneo.

Una **Rama (Branch)** permite divergir de la línea principal de desarrollo para trabajar en una nueva funcionalidad, corrección de errores o experimento de forma aislada. Esto garantiza que el código estable (usualmente en la rama `main`) nunca se vea comprometido por código en desarrollo incompleto.



5.1 Arquitectura de una Rama

Técnicamente, una rama en Git no es una copia de los archivos. Es simplemente un **puntero móvil** (una referencia ligera) que señala a un commit específico.

Cuando realizas un commit en una rama, el puntero de esa rama avanza automáticamente para apuntar al nuevo commit, mientras que los punteros de otras ramas permanecen estáticos.

- **HEAD:** Es un puntero especial que indica "dónde estás ahora mismo". Normalmente, el HEAD apunta al nombre de la rama en la que estás trabajando.

5.2 Gestión de Ramas

5.2.1 Creación de Ramas

Para crear una nueva rama, se utiliza el comando `branch`. Esto crea un nuevo puntero en el commit actual, pero **no** te cambia automáticamente a esa rama.

```
git branch nombre-de-la-rama
```

5.2.2 Cambio de Ramas (Switching)

Para moverte a otra rama (es decir, mover el HEAD y actualizar tu Directorio de Trabajo con los archivos de esa rama), históricamente se usaba `git checkout`. Sin embargo, las versiones modernas de Git (2.23+) introdujeron un comando más semántico y específico:

```
git switch nombre-de-la-rama
```

Para optimizar el flujo, es común crear la rama y cambiar a ella en un solo paso:

```
git switch -c nombre-de-la-rama
```

(Nota: El equivalente antiguo es `git checkout -b nombre-de-la-rama`).

5.2.3 Listado de Ramas

Para visualizar las ramas existentes y saber dónde se encuentra el HEAD (marcado con un asterisco `*`):

```
git branch
```

5.3 Fusión de Ramas (Merging)

Una vez completado el trabajo en una rama secundaria, el objetivo es integrar esos cambios de vuelta a la rama principal (o rama destino). Este proceso se denomina **Fusión** o **Merge**.

El flujo siempre es el mismo:

1. Te sitúas en la rama **destino** (aquella que recibirá los cambios, por ejemplo, `main`).
2. Ejecutas el comando de fusión indicando la rama **origen**.

```
git switch main  
git merge nombre-de-la-rama
```

Git gestiona la fusión de dos maneras distintas dependiendo de la historia del repositorio:

5.3.1 1. Fusión Fast-forward (Avance rápido)

Ocurre cuando la historia es lineal. Si la rama `main` no ha avanzado desde que creaste tu rama secundaria, Git simplemente "mueve el puntero" de `main` hacia adelante hasta alcanzar el último commit de la rama secundaria.

- **Característica:** No se crea un "commit de fusión". El historial se mantiene completamente lineal.

5.3.2 2. Fusión Recursiva (Merge Commit)

Ocurre cuando la historia ha divergido. Si la rama `main` ha recibido nuevos commits mientras tú trabajabas en tu rama secundaria, Git no puede simplemente mover el puntero.

En este caso, Git debe realizar una fusión de tres vías (3-way merge):

1. El último commit de la rama `main`.
2. El último commit de la rama secundaria.
3. El ancestro común de ambas ramas.

- **Resultado:** Git crea automáticamente un nuevo commit especial llamado **Merge Commit** que tiene dos padres. Este commit une las dos líneas de historia.

5.4 Eliminación de Ramas

Una vez que una rama ha sido fusionada correctamente y su código ya forma parte de `main`, la rama secundaria (el puntero) se vuelve redundante. Mantener ramas viejas ensucia el repositorio.

Para eliminar una rama local:

```
git branch -d nombre-de-la-rama
```

Git posee un mecanismo de seguridad: si intentas borrar una rama que contiene cambios que **no** han sido fusionados (y por tanto se perderían), el comando fallará y mostrará una advertencia.

Si deseas forzar el borrado de una rama (por ejemplo, un experimento fallido que quieres descartar), utiliza la bandera mayúscula `-D`:

```
git branch -D nombre-de-la-rama
```

6 Repositorios Remotos y Sincronización

Hasta este punto, todas las operaciones descritas se han realizado en el entorno local. Sin embargo, la potencia de Git como herramienta de colaboración reside en su capacidad para sincronizar el historial de cambios con otros ordenadores a través de una red.

Un **Repositorio Remoto** es una versión de tu proyecto alojada en internet o en otra red. Habitualmente, este "servidor" actúa como la fuente de verdad del proyecto. Las plataformas como GitHub, GitLab o Bitbucket proveen estos repositorios remotos junto con herramientas de gestión.

6.1 Protocolos de Conexión

Para que tu ordenador local se comuniquen con el servidor remoto, Git utiliza principalmente dos protocolos de transporte:

1. HTTPS:

- Utiliza la autenticación basada en usuario y contraseña (o *Personal Access Tokens*).
- Es más sencillo de configurar inicialmente, pero requiere revalidación frecuente si no se utiliza un gestor de credenciales.
- URL típica: `https://github.com/usuario/proyecto.git`

1. SSH (Secure Shell):

- Utiliza un par de claves criptográficas (pública y privada). La clave pública se sube al servidor (GitHub/GitLab) y la privada reside en tu ordenador.
- Es el **estándar profesional**. Una vez configurado, permite la conexión segura sin necesidad de introducir credenciales en cada operación.
- URL típica: `git@github.com:usuario/proyecto.git`

6.2 Vinculación de un Repositorio (`git remote`)

Si has iniciado un proyecto localmente con `git init`, necesitas decirle a Git dónde se encuentra su contraparte en la nube. Git gestiona estas conexiones mediante "alias" o nombres cortos.

El nombre estándar por convención para el servidor principal es `origin`.

Para vincular un repositorio remoto:

```
git remote add origin <url_del_repositorio>
```

Para verificar qué remotos están configurados:

```
git remote -v
```


La salida mostrará las direcciones de lectura (fetch) y escritura (push).

6.3 Obtención de un Proyecto Existente (`git clone`)

Si el proyecto ya existe en el servidor remoto y deseas descargarlo a tu máquina por primera vez (por ejemplo, al incorporarte a un equipo), no debes usar `git init`. El comando adecuado es `git clone`.

```
git clone <url_del_repositorio>
```

Este comando realiza tres acciones en un solo paso:

1. Crea una carpeta nueva con el nombre del proyecto.
2. Inicializa un directorio `.git` en su interior.
3. Añade automáticamente el remoto `origin` y descarga todo el historial y la rama principal.

6.4 Publicación de Cambios (`git push`)

El comando `push` se utiliza para cargar el contenido del repositorio local al repositorio remoto. Técnicamente, transfiere los commits que tienes en tu rama local pero que aún no existen en el servidor.

La sintaxis básica es:

```
git push <nombre-remoto> <nombre-rama>
```

6.4.1 Configuración del Upstream

La primera vez que subes una rama nueva (o la rama `main` recién creada), debes establecer una relación de rastreo (*upstream tracking*) entre tu rama local y la rama remota.

```
git push -u origin main
```

La bandera `-u` (o `--set-upstream`) le dice a Git: "Recuerda que mi rama local `main` está conectada a la rama `main` de `origin`". Gracias a esto, en futuras ocasiones podrás ejecutar simplemente `git push` sin argumentos.

6.5 Actualización y Descarga (`git fetch` vs `git pull`)

Cuando trabajas en equipo, el repositorio remoto avanzará mientras tú trabajas en local. Para obtener las actualizaciones de tus compañeros, existen dos estrategias que es vital diferenciar.

6.5.1 1. `git fetch` : La opción segura

Este comando descarga la información del remoto (nuevos commits, nuevas ramas), pero **no** modifica tus archivos de trabajo. Actualiza las referencias remotas (ej. `origin/main`), permitiéndote inspeccionar qué han hecho los demás antes de integrarlo.

```
git fetch origin
```

Es una operación de solo lectura para tu código local. Es seguro ejecutarlo en cualquier momento.

6.5.2 2. `git pull` : La opción directa

Este comando es, en realidad, un atajo (macro) que ejecuta dos comandos secuencialmente:

1. `git fetch` (descarga los datos).
2. `git merge` (fusiona los datos descargados en tu rama actual).

```
git pull origin main
```

Consideración Importante: Dado que `git pull` intenta fusionar código automáticamente, si hay cambios incompatibles entre tu versión local y la remota, **se producirá un conflicto de fusión** inmediatamente.

6.6 Flujo de Trabajo Remoto Estándar

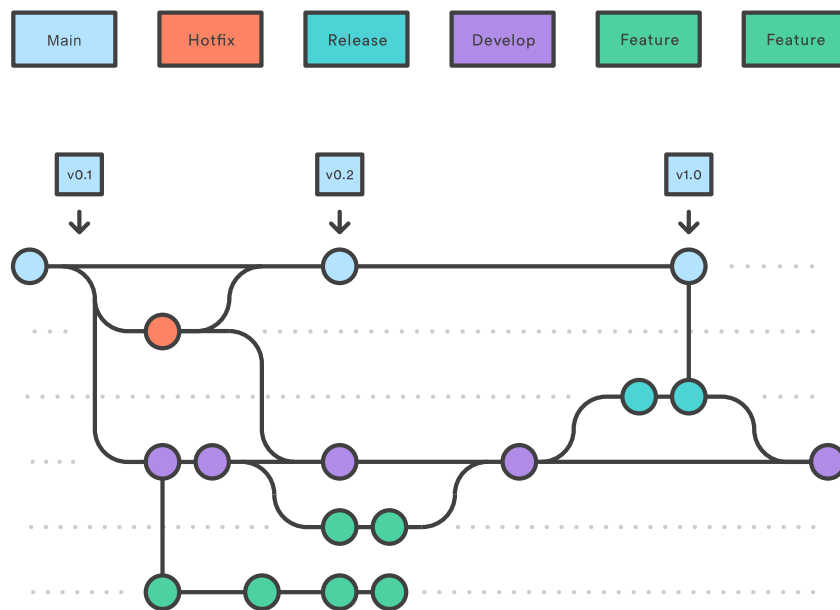
Para minimizar conflictos al trabajar con remotos, se recomienda seguir este orden antes de empezar a programar cada día o antes de hacer un push:

1. Bajar los últimos cambios: `git pull` .
2. Crear una rama para tu tarea: `git switch -c mi-funcionalidad` .
3. Trabajar y hacer commits.
4. Subir tu rama: `git push -u origin mi-funcionalidad` .

7 Estrategias de Flujo de Trabajo: Gitflow

Git es una herramienta agnóstica: no impone ninguna metodología sobre cómo usar las ramas. Sin embargo, en equipos de desarrollo profesional, "hacer lo que uno quiera" lleva al caos. Para solucionar esto, se utilizan **Flujos de Trabajo (Workflows)**: conjuntos de reglas y convenciones que dictan cómo deben interactuar las ramas.

El modelo más extendido y enseñado en la industria se conoce como **Gitflow**. Aunque existen alternativas más modernas (como *Trunk Based Development*), Gitflow ofrece una estructura rígida y clara ideal para gestionar ciclos de lanzamiento de software controlados.



7.1 Las Ramas Principales (Long-Lived Branches)

En Gitflow, el repositorio central mantiene dos ramas que existen infinitamente:

`main` 7.1.1 (anteriormente `master`)

- **Propósito:** Contiene el código de **producción**.
- **Estado:** Es sagrado. El código en esta rama debe estar libre de errores, probado y listo para ser usado por el usuario final en cualquier momento.
- **Regla de oro:** Nunca se realiza un commit directo sobre `main` . Solo recibe fusiones desde otras ramas (habitualmente `release` o `hotfix`).

7.1.2 `develop`

- **Propósito:** Es la rama de **integración**. Contiene el código de la próxima versión que se está desarrollando.
- **Estado:** Es relativamente estable, pero puede contener funcionalidades terminadas que aún no han salido a producción.
- **Origen:** Se crea a partir de `main` al inicio del proyecto.

7.2 Las Ramas de Soporte (Temporary Branches)

Estas ramas tienen un ciclo de vida limitado: se crean, cumplen una tarea específica, se fusionan y finalmente se eliminan.

7.2.1 Ramas de Funcionalidad (`feature/`)

Son donde los desarrolladores pasan la mayor parte del tiempo. Cada nueva funcionalidad (un botón, una API, un diseño) debe tener su propia rama.

- **Origen:** Salen de `develop` .
- **Destino:** Se fusionan de vuelta en `develop` .
- **Convención de nombre:** `feature/nombre-descriptivo` (ej. `feature/login-google`).

Flujo de trabajo:

```
# 1. Crear la rama desde develop
git switch develop
git switch -c feature/nueva-funcionalidad

# 2. Trabajar y hacer commits...

# 3. Fusionar de vuelta a develop (habitualmente mediante Pull Request)
git switch develop
git merge feature/nueva-funcionalidad
git branch -d feature/nueva-funcionalidad
```

7.2.2 Ramas de Lanzamiento (`release/`)

Cuando `develop` ha acumulado suficientes funcionalidades para una nueva versión (ej. versión 1.0), se crea una rama de `release`.

- **Propósito:** Preparar el código para producción. Aquí solo se permite corregir bugs menores y actualizar documentación o números de versión. **No se añaden funcionalidades nuevas.**
- **Origen:** Salen de `develop`.
- **Destino:** Se fusionan en `main` **Y** en `develop`.
- **Convención de nombre:** `release/v1.0`.

Este paso permite que un equipo termine de pulir la versión 1.0 mientras otro equipo empieza a trabajar en funcionalidades para la versión 1.1 en `develop`.

7.2.3 Ramas de Parche (`hotfix/`)

Son las únicas ramas que pueden nacer directamente de `main`. Se utilizan para situaciones de emergencia: un error crítico en producción que debe arreglarse inmediatamente sin esperar a la próxima versión planificada.

- **Origen:** Salen de `main`.
- **Destino:** Se fusionan en `main` **Y** en `develop` (para asegurar que el error no reaparezca en la próxima versión).
- **Convención de nombre:** `hotfix/error-critico-pagos`.

7.3 Resumen del Ciclo de Vida en Gitflow

Para visualizar cómo se mueven los datos en este modelo:

1. Se inicializa el proyecto con `main` .
2. Se crea `develop` a partir de `main` .
3. Los desarrolladores crean ramas `feature/` desde `develop` para trabajar.
4. Al terminar, las `feature/` se integran en `develop` .
5. Cuando estamos listos para lanzar, creamos una rama `release/` desde `develop` .
6. Al finalizar el release, se fusiona a `main` (se crea una nueva versión) y se actualiza `develop` con los cambios de pulido.
7. Si surge una emergencia en `main` , se usa una rama `hotfix/` .

7.4 Consideraciones Técnicas: El "Merge Commit"

En Gitflow, es habitual forzar la creación de un *Merge Commit* (usando la bandera `--no-ff` en el merge), incluso si se podría hacer un *Fast-forward*.

¿Por qué? Porque esto permite mantener agrupados en el historial todos los commits que pertenecían a una misma funcionalidad. Esto facilita enormemente revertir una funcionalidad completa si algo sale mal en el futuro, manteniendo la trazabilidad histórica del proyecto.

8 Colaboración y Resolución de Conflictos

En un entorno profesional, múltiples desarrolladores trabajan simultáneamente sobre la misma base de código. Aunque Git es extremadamente eficiente fusionando cambios automáticos (por ejemplo, si dos personas editan archivos distintos), existen situaciones donde la lógica del software no puede tomar decisiones autónomas.

Este capítulo detalla la naturaleza técnica de los conflictos de fusión y el protocolo estándar para la integración de código en equipos: el *Pull Request*.

8.1 La Naturaleza del Conflicto

Un **conflicto de fusión** ocurre cuando Git intenta combinar dos ramas, pero encuentra instrucciones contradictorias que no puede resolver algorítmicamente.

Las causas más frecuentes son:

1. Dos desarrolladores han modificado las **mismas líneas** del mismo archivo de manera diferente.
2. Un desarrollador ha eliminado un archivo mientras otro lo estaba modificando.

Cuando esto sucede, Git entra en un estado de **pausa**. El proceso de fusión (`merge` o `pull`) se detiene y requiere intervención humana para decidir qué versión del código es la correcta.

8.2 Identificación y Lectura de Conflictos

Al ejecutarse un comando conflictivo (ej. `git merge`), la terminal devolverá un error explícito:

```
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

En este estado, el archivo afectado (`index.html`) es modificado físicamente por Git, inyectando unos **marcadores de conflicto** que delimitan las versiones enfrentadas.

Es imperativo abrir el archivo en el editor de código para examinar estos bloques:

```
<<<<<< HEAD
<button>Iniciar Sesión</button>
=====
<button>Log In</button>
>>>>>> rama-login
```

- <<<<<< **HEAD** : Indica el inicio de los cambios presentes en tu rama actual (donde estás situado).
- ===== : Es el separador. Todo lo que está encima pertenece a tu versión; todo lo que está debajo, a la versión entrante.
- >>>>>> **rama-login** : Indica el fin del bloque conflictivo y muestra de qué rama provienen los cambios rivales.

8.3 El Protocolo de Resolución

Resolver un conflicto es un proceso manual de edición. Git no "sabe" qué botón es el correcto; tú, como desarrollador, debes dictar la solución final.

8.3.1 Paso 1: Edición

Debes editar el archivo para dejarlo exactamente como quieres que se guarde en la historia. Esto implica obligatoriamente:

1. **Borrar** los marcadores de Git (<<<<<< , ===== , >>>>>>).
2. **Elegir** una de las dos opciones, combinar ambas, o escribir un código completamente nuevo.

Ejemplo de resolución (decidimos quedarnos con la versión en inglés):

```
<button>Log In</button>
```

8.3.2 Paso 2: Marcado (`git add`)

Una vez guardado el archivo limpio, debes indicar a Git que el conflicto ha sido resuelto. Para ello, se utiliza el comando de preparación:

```
git add index.html
```

Al añadir el archivo al *Staging Area*, Git entiende que has aceptado la versión actual del archivo como la solución definitiva.

8.3.3 Paso 3: Finalización (`git commit`)

Para concluir el proceso de fusión que quedó en pausa, simplemente ejecuta:

```
git commit
```

Generalmente no es necesario añadir la bandera `-m`, ya que Git generará automáticamente un mensaje tipo *"Merge branch 'rama-login'"* indicando que se trata de un commit de resolución de conflictos.

Nota de Seguridad: Si te encuentras en medio de una fusión conflictiva y decides que no quieres continuar (por ejemplo, porque te faltan conocimientos para resolverlo), puedes abortar la operación y volver al estado previo al intento de fusión con: `git merge --abort`

8.4 El Modelo de Pull Request (PR)

En equipos profesionales, rara vez se hace un `git merge` directo en la terminal hacia la rama `main` o `develop`. En su lugar, se utiliza un mecanismo de las plataformas de alojamiento (GitHub/GitLab) llamado **Pull Request (PR)** o *Merge Request*.

Un Pull Request es una petición formal para integrar cambios. No es un comando de Git, sino un proceso administrativo que permite:

1. **Code Review (Revisión de Código):** Otros miembros del equipo pueden leer tus cambios, comentar línea por línea y sugerir mejoras antes de que el código toque la rama principal.
2. **Integración Continua (CI):** Se ejecutan pruebas automáticas para asegurar que los nuevos cambios no rompen el proyecto.

8.4.1 Flujo de Trabajo con PR

1. **Push:** Subes tu rama de funcionalidad al servidor remoto.

```
git push -u origin feature/mi-rama
```

1. **Apertura:** En la interfaz web (GitHub/GitLab), creas el Pull Request comparando tu rama contra `develop` (o `main`).
2. **Discusión:** El equipo revisa el código. Si solicitan cambios, los realizas en tu ordenador local y vuelves a hacer `push` sobre la misma rama. El PR se actualiza automáticamente.
3. **Aprobación y Merge:** Una vez aprobado, se pulsa el botón de "Merge" en la interfaz web.
4. **Sincronización:** En tu ordenador local, vuelves a la rama principal y descargas la fusión que acaba de ocurrir en remoto.

```
git switch develop  
git pull origin develop
```

9 Recuperación de Errores y Mantenimiento

En el desarrollo de software, el error es inevitable. Se cometen faltas de ortografía en los mensajes de commit, se olvidan archivos en la zona de preparación o se trabaja en la rama equivocada. Git ofrece herramientas potentes para corregir estos deslices.

Sin embargo, estas herramientas de "reescritura de historia" tienen una regla de oro: **Solo deben usarse en commits que no hayan sido subidos (push) todavía al repositorio remoto**. Reescribir la historia compartida puede desincronizar el trabajo de todo el equipo.

9.1 Refinando el Último Commit (`--amend`)

El escenario más común es realizar un commit y darse cuenta inmediatamente de que se olvidó incluir un archivo o que el mensaje tiene un error tipográfico. En lugar de crear un nuevo commit de corrección que ensucie el historial, podemos "editar" el anterior.

9.1.1 Modificar el mensaje

Si solo deseas cambiar el texto del último commit:

```
git commit --amend -m "Nuevo mensaje corregido"
```

9.1.2 Añadir archivos olvidados

Si olvidaste añadir un archivo al commit anterior:

1. Añade el archivo al *Staging Area*: `git add archivo_olvidado.txt`
2. Ejecuta el comando de enmienda sin mensaje (para reutilizar el anterior) o con uno nuevo:

```
git commit --amend --no-edit
```

Nota Técnica: En realidad, Git no "edita" el commit antiguo. Lo que hace es crear un **nuevo commit** (con un nuevo hash SHA-1) que reemplaza al anterior y mueve el puntero HEAD.

9.2 El Botón de "Deshacer": `git reset`

El comando `git reset` es una herramienta de precisión para mover el puntero `HEAD` hacia atrás en el tiempo. Al mover el puntero, podemos deshacer commits. Existen tres modos de operación que determinan qué ocurre con los archivos que se "sueltan" al retroceder.

Supongamos que queremos deshacer el último commit (volver al estado anterior):

9.2.1 Soft Reset (`--soft`)

Mueve el HEAD al commit anterior, pero **mantiene los cambios en el Staging Area**.

- **Uso:** "Hice commit, pero quiero deshacerlo para añadir más cosas y volver a commitear todo junto".
- **Comando:** `git reset --soft HEAD~1` (*HEAD~1 significa "un paso atrás desde la cabecera"*).

9.2.2 Mixed Reset (`--mixed`)

Es el comportamiento por defecto. Mueve el HEAD hacia atrás y **mueve los cambios al Working Directory** (quitándolos del Staging Area).

- **Uso:** "Hice commit, pero quiero seguir trabajando en estos archivos antes de prepararlos de nuevo".
- **Comando:** `git reset HEAD~1`

9.2.3 Hard Reset (`--hard`)

Peligro. Mueve el HEAD hacia atrás y **elimina todos los cambios** del Staging Area y del Working Directory.

- **Uso:** "Quiero destruir todo lo que he hecho en el último commit y dejarlo todo como estaba antes de empezar".
- **Comando:** `git reset --hard HEAD~1`

Advertencia: `git reset --hard` es destructivo. Los cambios no guardados en otro commit se perderán permanentemente.

9.3 Almacenamiento Temporal: `git stash`

Imagina que estás trabajando en una funcionalidad compleja (rama `feature`) y tienes el directorio de trabajo "sucio" (archivos a medio editar). De repente, te piden arreglar un bug urgente en otra rama.

Git no te dejará cambiar de rama (`switch`) si tienes cambios que entran en conflicto con la rama de destino. Tienes dos opciones: hacer un commit de un trabajo a medias (mala práctica) o usar **Stash**.

El **Stash** es una pila de almacenamiento temporal (un "cajón desastre") donde puedes guardar cambios sin hacer commit.

9.3.1 Guardar en el Stash

Para limpiar tu mesa de trabajo y guardar los cambios actuales:

```
git stash
```

Ahora tu directorio de trabajo estará limpio (como en el último commit) y podrás cambiar de rama tranquilamente.

9.3.2 Recuperar del Stash

Una vez terminado el arreglo urgente, vuelves a tu rama original y recuperas tu trabajo:

```
git stash pop
```

Este comando saca los últimos cambios guardados en la pila y los aplica a tu directorio actual.

9.3.3 Listar cambios guardados

Si has guardado varias cosas, puedes ver la lista con:

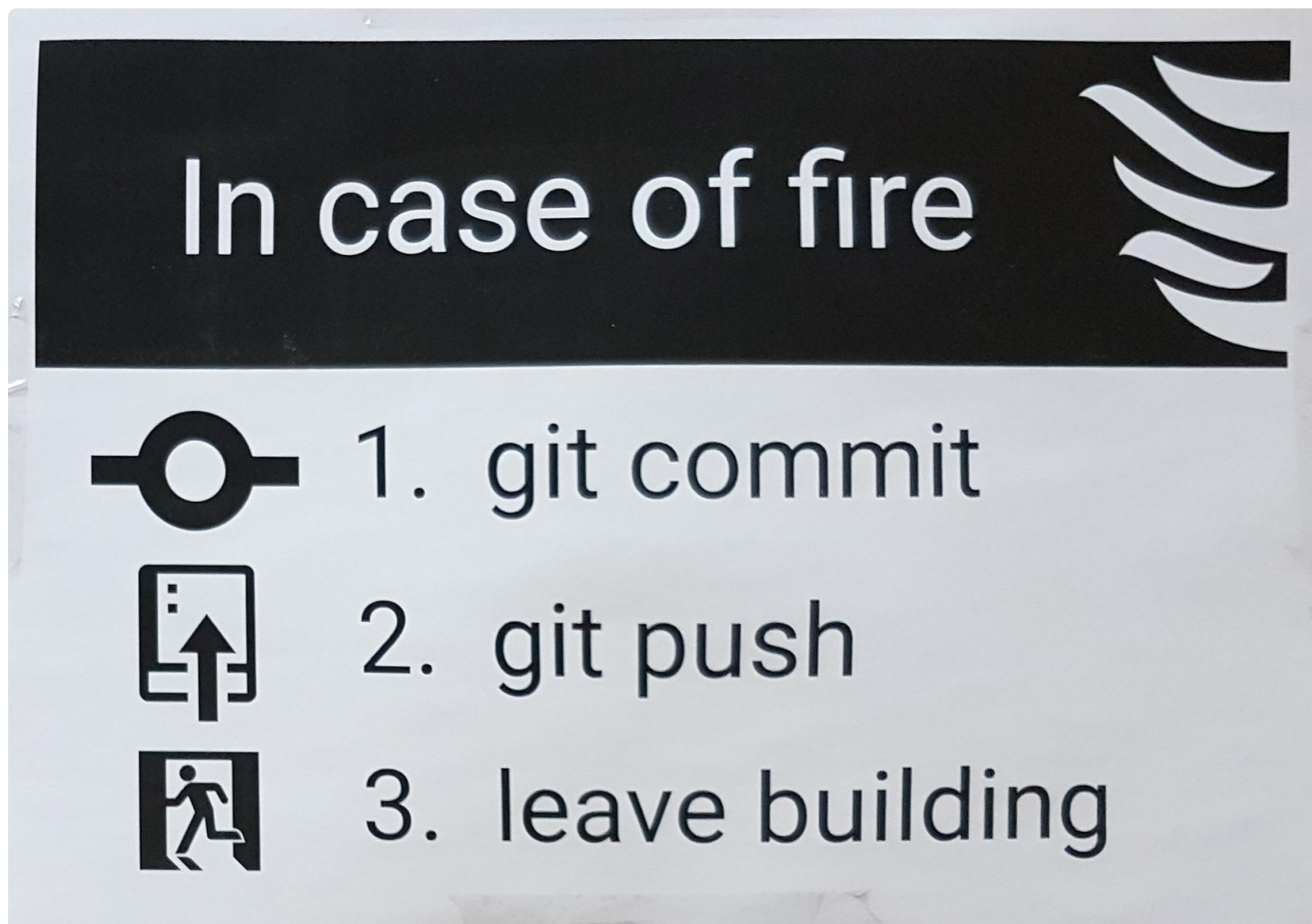
```
git stash list
```

9.4 Resumen de Comandos de Emergencia

Situación	Solución
Commit con mensaje erróneo	<code>git commit --amend</code>
Olvidé añadir un archivo al commit	<code>git add .</code> + <code>git commit --amend</code>
Quiero deshacer el commit pero guardar los cambios	<code>git reset --soft HEAD~1</code>
Quiero destruir mis cambios locales recientes	<code>git restore .</code>
Quiero borrar el último commit y sus cambios	<code>git reset --hard HEAD~1</code>
Necesito cambiar de rama con trabajo a medias	<code>git stash</code> y luego <code>git stash pop</code>

10 Glosario y Hoja de Referencia Rápida

Este capítulo final actúa como un compendio resumido de todo el material tratado en la guía. Su objetivo es servir como recurso de consulta rápida durante el desarrollo diario y como material de repaso para la preparación de entrevistas técnicas.



10.1 Glosario de Términos

Es fundamental dominar la terminología precisa para comunicarse eficazmente con otros desarrolladores.

Término	Definición Técnica
Repositorio (Repo)	Base de datos donde Git almacena el historial, los objetos y las referencias del proyecto.
Commit	Una instantánea (<i>snapshot</i>) inmutable del proyecto en un momento específico. Posee un identificador único (SHA-1).
Staging Area (Index)	Zona intermedia donde se preparan y organizan los cambios antes de ser confirmados en un commit.
Working Directory	Los archivos actuales en tu sistema de archivos local donde editas el código.
HEAD	Puntero que indica la posición actual en la historia (generalmente apunta a la rama activa).
Rama (Branch)	Puntero móvil hacia un commit. Permite el desarrollo de líneas paralelas de trabajo.
Merge (Fusión)	Acción de combinar la historia de dos ramas distintas. Puede resultar en un <i>Merge Commit</i> o un <i>Fast-forward</i> .
Remote	Referencia a una versión del repositorio alojada en otro servidor (ej. GitHub, GitLab).
Pull Request (PR)	Mecanismo (ajeno a Git, propio de plataformas de hosting) para proponer cambios y solicitar revisión de código antes de una fusión.
Fork	Una copia personal de un repositorio ajeno en tu cuenta de GitHub/GitLab. No es un comando de Git.
Upstream	Referencia a la rama original en el repositorio remoto con la que la rama local está sincronizada.

10.2 Cheatsheet de Comandos

A continuación se listan los comandos más frecuentes clasificados por su función en el ciclo de desarrollo.

10.2.1 Configuración e Inicio

Comando	Descripción
<code>git init</code>	Inicializa un nuevo repositorio Git en el directorio actual.
<code>git clone <url></code>	Descarga un repositorio remoto completo a tu máquina.
<code>git config --global user.name "<nombre>"</code>	Define el nombre de autor para los commits.
<code>git config --global user.email "<email>"</code>	Define el correo de autor para los commits.
<code>git config --list</code>	Muestra toda la configuración actual de Git.

10.2.2 Ciclo Básico (Local)

Comando	Descripción
<code>git status</code>	Muestra el estado de los archivos (modificados, preparados, sin seguimiento). Usar frecuentemente.
<code>git add <archivo></code>	Añade un archivo específico al Staging Area.
<code>git add .</code>	Añade todos los archivos modificados y nuevos al Staging Area.
<code>git commit -m "<mensaje>"</code>	Confirma los cambios del Staging Area con un mensaje descriptivo.
<code>git commit --amend</code>	Permite modificar el mensaje o añadir archivos al último commit realizado.

10.2.3 Gestión de Ramas

Comando	Descripción
<code>git branch</code>	Lista las ramas locales y marca la activa con un asterisco.
<code>git branch <nombre></code>	Crea una nueva rama, pero no cambia a ella.
<code>git switch <nombre></code>	Cambia el directorio de trabajo a la rama especificada.
<code>git switch -c <nombre></code>	Crea una rama nueva y cambia a ella inmediatamente.
<code>git merge <rama></code>	Fusiona la rama especificada dentro de la rama actual.
<code>git branch -d <rama></code>	Elimina una rama local (solo si ya ha sido fusionada).

10.2.4 Sincronización (Remoto)

Comando	Descripción
<code>git remote -v</code>	Muestra los repositorios remotos vinculados y sus URL.
<code>git fetch <remoto></code>	Descarga los últimos cambios del remoto sin fusionarlos (seguro).
<code>git pull <remoto> <rama></code>	Descarga los cambios y los fusiona en la rama actual (fetch + merge).
<code>git push <remoto> <rama></code>	Sube los commits locales al repositorio remoto.
<code>git push -u origin <rama></code>	Sube la rama y establece el rastreo (<i>upstream</i>) para futuros push simples.

10.2.5 Inspección y Deshacer

Comando	Descripción
<code>git log</code>	Muestra el historial completo de commits.
<code>git log --oneline --graph</code>	Muestra el historial resumido y visualiza la estructura de ramas.
<code>git diff</code>	Muestra las diferencias exactas (código) de los archivos modificados pero no preparados.
<code>git restore <archivo></code>	Descarta cambios en el directorio de trabajo (vuelve al estado del último commit).
<code>git restore --staged <archivo></code>	Saca un archivo del Staging Area, pero mantiene los cambios en el directorio.
<code>git reset --soft HEAD~1</code>	Deshace el último commit, pero mantiene los cambios listos para commitear de nuevo.
<code>git reset --hard HEAD~1</code>	Destruyivo. Elimina el último commit y todos los cambios asociados.

10.2.6 Utilidades (Stash)

Comando	Descripción
<code>git stash</code>	Guarda temporalmente los cambios actuales en una pila para limpiar el directorio.
<code>git stash pop</code>	Recupera y aplica los últimos cambios guardados en el stash.
<code>git stash list</code>	Muestra la lista de cambios guardados temporalmente.